

COMP2522

Lesson 8

IO, Path, Files, Scanner, Streaming

File Input/Output

- There are many classes and techniques to work with file IO
- The newest, cleanest ways involve the NIO ("new IO") and NIO2 classes
- The **Path** interface represents a location in the file system (files or directories).
- The **Files** class is a utility class to work with files and directories. It provides all static methods to read, write, create, delete, copy, and move files.
- Path and Files are platform independent; they handle file system operations across different operating systems seamlessly.

Path and Files

- <https://docs.oracle.com/en/java/javase/22/docs/api/java.base/java.nio.file/Path.html>
- <https://docs.oracle.com/en/java/javase/22/docs/api/java.base/java.nio.file/Files.html>

Path

- Always use `Paths.get()` to construct file paths instead of hardcoding paths.
- `Path path = Paths.get("data.txt");`
- `// Works on all OS`
- `Path path = Paths.get("src", "resources", "file.txt");`
- `Path path = Paths.get("src", "resources", "data", "2024", "reports", "file.txt");`
- Avoid hardcoded separators: Don't use `\` or `/` directly; `System.lineSeparator()` is better, but `Paths.get()` can manage paths alone.

Creating Files (and Directories)

- Creating a File:
- `Path path = Paths.get("newFile.txt");`
- `Files.createFile(path);` // Creates a new file

- Best Practice: Check if the file exists first.
- `if (Files.notExists(path)) {`
- `Files.createFile(path);`
- `}`
- Creating a Directory:
- `Path dirPath = Paths.get("newDir");`
- `Files.createDirectory(dirPath);`

Nested Directories

- Creating Nested Directories:
- `Files.createDirectories(Paths.get("dir/subdir"));`
- NOTE: the forward slash there is perfectly fine to use; forward slashes (/) and backward slashes (\) are automatically handled by the `Paths.get()` method to ensure platform independence
- NOTE: if "dir" doesn't exist in the code above, this method will also create it. There is a `createDirectory()` (singular) method which would NOT; it would throw an exception if the parent directory doesn't exist

Reading All (Small) Text File Lines at Once

With files smaller than *about* 10MB, we can read all data into a list at once.

```
try
{
    Final List<String> lines;
    lines = Files.readAllLines(path);
}
catch (final IOException e)
{
    e.printStackTrace();
}
```

Very Large Files, and try-with-resources

- Instead of loading all lines into memory, read huge files line by line with `BufferedReader`.
- Best Practice: Use **try-with-resources** to automatically close streams.
- ```
try (BufferedReader reader = Files.newBufferedReader(path))
{
 String line;
 while ((line = reader.readLine()) != null)
 {
 System.out.println(line);
 }
}
catch (final IOException e)
{
 e.printStackTrace();
}
```



# Writing/Appending Text to a File

- Writing to a File:
- `List<String> lines = Arrays.asList("First line", "Second line");`
- The next instruction overwrites the files if it exists:
- `Files.write(path, lines, StandardOpenOption.CREATE);`
- `Files.writeString(path, "one string, one line", StandardOpenOption.CREATE);`
- Best Practice: Use `StandardOpenOption` to control file behavior (e.g., prevent accidental overwrite).
- `Files.write(path, lines, StandardOpenOption.CREATE, StandardOpenOption.APPEND);`
- Appending Text to an existing file:
- `Files.write(path, lines, StandardOpenOption.APPEND);`

# Moving Files and Directories

- Moving a File:
  - `Path source = Paths.get("file.txt");`
  - `Path target = Paths.get("newDir/file.txt");`
  - `Files.move(source, target);`
  - Best Practice: Check if the target file already exists, handle conflicts.
- Overwriting Target:
  - `Files.move(source, target, StandardCopyOption.REPLACE_EXISTING);`
- Moving a Directory:
  - `Path dirSource = Paths.get("dir");`
  - `Path dirTarget = Paths.get("newDir");`
  - `Files.move(dirSource, dirTarget);`

# Copying Files and Directories

- Copying a File:
- `Path source = Paths.get("file.txt");`
- `Path target = Paths.get("copy.txt");`
- `Files.copy(source, target);`
- Best Practice: Use `StandardCopyOption` to handle overwriting and copying attributes.
- `Files.copy(source, target, StandardCopyOption.REPLACE_EXISTING);`
- Copying a Directory: Requires recursive copy (not simple with `Files.copy`).
- Best Practice: Use a method to recursively copy directory contents (advanced topic).

# Deleting Files and Directories

- Deleting a File:
- `Files.delete(Paths.get("file.txt"));`
- Best Practice: Check if the file exists before deleting, handle exceptions (e.g., `NoSuchFileException`).
- `try {`
- `Files.delete(path);`
- `} catch (final NoSuchFileException e) {`
- `System.out.println("File not found");`
- `}`
- Deleting a Directory: Must be empty before deletion.
- `Files.delete(Paths.get("emptyDir"));`

# Reading/Writing Binary Files

```
import java.nio.file.*;
import java.io.IOException;

public class BinaryFileExample {
 public static void main(String[] args) throws IOException {
 Path path = Paths.get("binaryfile.bin");

 // Data to write (a simple byte array)
 byte[] dataToWrite = { 1, 2, 3, 4, 5 };

 // Writing binary data
 Files.write(path, dataToWrite);

 // Reading binary data
 byte[] dataRead = Files.readAllBytes(path);

 // Output read data to verify
 for (byte b : dataRead) {
 System.out.print(b + " ");
 }
 }
}
```

See the next slide for a real-life application of this: copying an image from one file to another

```
import java.nio.file.*;
import java.io.IOException;
```

```
public class CopyImageExample {
 public static void main(final String[] args) throws IOException {
 // Path to the original image (source)
 Path source = Paths.get("sourceImage.jpg");

 // Path where the image will be copied (destination)
 Path destination = Paths.get("copiedImage.jpg");

 // Copy the binary image file
 Files.copy(source, destination, StandardCopyOption.REPLACE_EXISTING);

 // Verify by reading a few bytes from the copied file
 byte[] dataRead = Files.readAllBytes(destination);

 // Output the first few bytes to ensure the file was copied correctly
 System.out.println("First 5 bytes of the copied file:");
 for (int i = 0; i < 5; i++) {
 System.out.print(dataRead[i] + " ");
 }
 }
}
```

# copying an image from one file to another

# Best Practices

- Always use Path for file system operations (modern, flexible).
- Handle large files using streams (don't load everything into memory).
- Use try-with-resources to ensure resources are closed automatically.
- Use StandardOpenOption to avoid accidental overwriting.
- Always check if files/directories exist before operations like move, delete, and copy.

```

public class Game {
 public static void main(final String[] args) {
 final Random rand = new Random();
 int target = rand.nextInt(1, 11); // Random number between 1 and 10
 System.out.println("Guess between 1 and 10, or 'Q' to quit (in 'input.txt').");
 try {
 // Read all guesses from the file once
 final List<String> lines = Files.readAllLines(Paths.get("input.txt"));
 for (String input : lines) {
 input = input.trim();
 if (input.equalsIgnoreCase("Q")) {
 System.out.println("Game ended by user. Goodbye!");
 break;
 }
 }
 try {
 // Attempt to parse the input as an integer
 int guess = Integer.parseInt(input);
 if (guess < 1 || guess > 10) {
 System.out.println("Out of bounds! Guess between 1 and 10, not " + guess);
 } else if (guess < target) {
 System.out.println("Higher than " + guess);
 } else if (guess > target) {
 System.out.println("Lower than " + guess);
 } else {
 System.out.println("Correct! The number was " + target);
 target = rand.nextInt(10) + 1; // Generate a new target
 System.out.println("New number generated. Guess or type 'Q' to quit.");
 }
 } catch (final NumberFormatException e) {
 System.out.printf("Invalid input (%s)! Please enter a number." +
 System.lineSeparator(), input);
 }
 } catch (final IOException e) {
 System.out.println("Error reading file: " + e.getMessage());
 }
 System.out.println("The end.");
 }
}

```

# Random, File, Game

Sample input.txt and sample output by Game

```

1
2
3
4
5
6
7
8
9
10
11
12
hello
whatever
quit
5
4
3
2
1
8
7
5
4
1
55
4
q
5
4

```

```

Guess a number between 1 and 10, or type 'Q' to quit (enter your guesses in 'input.txt').
Higher than 1
Higher than 2
Higher than 3
Higher than 4
Higher than 5
Higher than 6
Higher than 7
Higher than 8
Higher than 9
Correct! The number was 10
New number generated. Continue guessing or type 'Q' to quit.
Out of bounds! Guess a number between 1 and 10, not 11
Out of bounds! Guess a number between 1 and 10, not 12
Invalid input (hello)! Please enter a number.
Invalid input (whatever)! Please enter a number.
Invalid input (quit)! Please enter a number.
Correct! The number was 5
New number generated. Continue guessing or type 'Q' to quit.
Higher than 4
Higher than 3
Higher than 2
Higher than 1
Lower than 8
Correct! The number was 7
New number generated. Continue guessing or type 'Q' to quit.
Higher than 5
Higher than 4
Higher than 1
Out of bounds! Guess a number between 1 and 10, not 55
Higher than 4
Game ended by user. Goodbye!
The end.

```



# Random, Keyboard, Game

```
public class Game3 {
 public static void main(final String[] args) {
 final Random rand = new Random();
 int target = rand.nextInt(1, 11);
 System.out.println("Guess a number between 1 and 10, or type 'Q' to quit.");

 // Initialize Scanner to read from the keyboard
 try (Scanner scanner = new Scanner(System.in)) {
 while (true) {
 System.out.print("Enter your guess: ");
 String input = scanner.nextLine().trim();
 // Check if the user wants to quit
 if (input.equalsIgnoreCase("Q")) {
 System.out.println("Game ended by user. Goodbye!");
 break;
 }
 try {
 // Attempt to parse the input as an integer
 int guess = Integer.parseInt(input);
 if (guess < 1 || guess > 10) {
 System.out.println("Out of bounds! Guess a number between 1 and 10, not " + guess);
 } else if (guess < target) {
 System.out.println("Higher than " + guess);
 } else if (guess > target) {
 System.out.println("Lower than " + guess);
 } else {
 System.out.println("Correct! The number was " + target);
 target = rand.nextInt(1, 11); // Generate a new target
 System.out.println("New number generated. Continue guessing or type 'Q' to quit.");
 }
 } catch (final NumberFormatException e) {
 System.out.printf("Invalid input (%s)! Please enter a number." + System.lineSeparator(), input);
 }
 }
 }
 System.out.println("The end.");
 }
}
```